

Trilha IA Engineering

Plano completo para nível avançado/sênior

Foco	IA aplicada a produtos reais com LLMs, RAG, agentes, fine-tuning, LLMOps, segurança e governança.
Formato	Trilha progressiva com fundamentos, prática, arquitetura, produção e liderança técnica.
Resultado esperado	Preparação para atuar como AI Engineer avançado/sênior.
Observação	Documento formatado a partir do conteúdo anexado.
Versão	Reformatação editorial - 07/06/2026

Material técnico · Plano de estudos · Arquitetura de IA · Produção e governança

Sumário do documento

Estrutura principal da trilha formatada neste PDF, com numeração de páginas e hierarquia de seções.

Trilha IA Engineering — Plano completo para nível avançado/sênior	4
1. Análise inicial da trilha	4
2. Explicação técnica por grande área	5
2.1 Fundamentos de IA	5
2.2 Machine Learning clássico	6
2.3 Deep Learning	8
2.4 LLMs	10
1. Mini-transformer educacional	11
2. Aplicação com LLM para análise de documentos técnicos	11
2.5 Prompt Engineering avançado	12
2.6 RAG avançado	13
2.7 Fine-tuning	15
2.8 Agentes	17
2.9 LLMOps	18
2.10 Segurança e governança de IA	20
3. Plano de estudos em 6 módulos	21
Módulo 1 — Fundamentos	21
Módulo 2 — Aplicação prática com Machine Learning clássico	22
Módulo 3 — Aprofundamento em Deep Learning e LLMs	24
Módulo 4 — Arquitetura, Prompt Engineering, RAG e Fine-tuning	26
Módulo 5 — Produção, performance, segurança e escalabilidade	28
Módulo 6 — Domínio sênior e liderança técnica	31
4. Critérios de domínio técnico por área	33
5. Aplicação prática em sistemas reais	34
6. Cronograma sugerido	37

7. Projetos recomendados para portfólio	38
Projeto 1 — ML Risk Scoring API	38
Projeto 2 — Document Intelligence com LLM	39
Projeto 3 — RAG DevCore	39
Projeto 4 — Fine-tuned Technical QA	39
Projeto 5 — Agente Técnico Controlado	40
Projeto 6 — LLMOps Dashboard	40
8. Checklist de domínio técnico	40
9. Indicadores de prontidão para nível avançado/sênior	42
Resumo final da trilha	43

Trilha IA Engineering — Plano completo para nível avançado/sênior

1. Análise inicial da trilha

Tema central

A trilha é focada em IA Engineering, isto é, a engenharia aplicada ao desenvolvimento de produtos com Inteligência Artificial, Machine Learning, Deep Learning, LLMs, RAG, agentes, fine-tuning, observabilidade, segurança e operação em produção.

O foco não é apenas “usar IA”, mas projetar sistemas confiáveis, avaliáveis, seguros, escaláveis e economicamente viáveis com IA.

Objetivo principal

Formar um profissional capaz de:

- entender fundamentos matemáticos, estatísticos e computacionais de IA;
- construir modelos clássicos de Machine Learning;
- compreender Deep Learning e Transformers;
- integrar LLMs em aplicações reais;
- construir RAGs com avaliação, segurança e controle de alucinação;
- criar agentes com ferramentas, memória e permissões;
- operar aplicações de IA em produção com LLMOps;
- aplicar governança, LGPD, auditoria e segurança em sistemas de IA;
- tomar decisões arquiteturais de custo, risco, performance, privacidade e escalabilidade.

Competências técnicas a desenvolver

Um AI Engineer sênior precisa combinar competências de várias áreas:

Área	Competência esperada
IA/ML	Saber modelar problemas, treinar, avaliar e comparar modelos.
Deep Learning	Entender redes neurais, treinamento, otimização e inferência.
LLMs	Entender tokenização, atenção, janela de contexto, grounding, alucinação e parâmetros de geração.
RAG	Projetar pipelines de ingestão, busca, reranking, autorização, citação e avaliação.
Fine-tuning	Preparar dataset, treinar modelos pequenos, comparar resultados e controlar custo.
Agentes	Criar fluxos com ferramentas, memória, planejamento, limites e segurança.
LLMOps	Versionar prompts, datasets, modelos, avaliações, custos, latência e qualidade.
Segurança	Mitigar prompt injection, vazamento de dados, abuso de ferramentas e uso indevido.
Arquitetura	Decidir entre prompt, RAG, fine-tuning, agente, modelo próprio ou API externa.
Produto	Medir valor, custo por requisição, qualidade percebida e risco operacional.

Classificação por nível

Nível	Conteúdos principais
Básico	IA, ML, NLP, Computer Vision, métricas, overfitting, bias/variance, modelos supervisionados e não supervisionados.
Intermediário	Regressão, classificação, árvores, ensembles, pipelines, cross-validation, feature engineering, MLflow, métricas de avaliação.
Avançado	Deep Learning, Transformers, LLMs, prompt engineering, RAG, fine-tuning, agentes, serving, avaliação automática.
Sênior	LLMOps, arquitetura de IA em produção, segurança, governança, custo, observabilidade, auditoria, estratégia técnica e liderança.

2. Explicação técnica por grande área

2.1 Fundamentos de IA

O que é

Inteligência Artificial é o campo que busca construir sistemas capazes de executar tarefas associadas a percepção, linguagem, raciocínio, decisão, previsão, geração de conteúdo e automação. Dentro dela existem áreas como Machine Learning, Deep Learning, NLP, Computer Vision, Generative AI, LLMs, agentes e sistemas especialistas.

Para que serve

Serve para criar sistemas que:

- classificam dados;
- fazem previsões;
- detectam padrões;
- entendem texto;
- geram conteúdo;
- automatizam decisões;
- recomendam ações;
- extraem conhecimento de documentos;
- interagem com usuários por linguagem natural.

Por que é importante

IA deixou de ser apenas pesquisa e virou componente central de produto. Hoje, sistemas reais usam IA para recomendação, detecção de fraude, análise de documentos, suporte ao cliente, busca semântica, automação de atendimento, copilotos internos, classificação de tickets e geração de insights.

Onde é usado em projetos reais

- API que classifica chamados de suporte.
- Sistema de recomendação de produtos.
- Chatbot com RAG sobre base documental.

- Detecção de anomalias em logs.
- Análise automática de contratos.
- OCR + NLP para extração de dados.
- Copiloto interno para times jurídicos, técnicos ou comerciais.

Problemas que resolve

- Alto volume de dados não estruturados.
- Busca textual limitada.
- Processos repetitivos.
- Falta de escala em atendimento.
- Classificação manual lenta.
- Dificuldade de extrair conhecimento de documentos.

Erros comuns

- Achar que IA resolve qualquer problema.
- Usar LLM onde uma regra simples resolveria.
- Não medir qualidade.
- Não separar treino, validação e teste.
- Ignorar viés, privacidade e segurança.
- Confundir protótipo com sistema de produção.
- Usar IA sem observabilidade e sem avaliação.

Conhecimentos prévios necessários

- Python.
- Estatística básica.
- Álgebra linear básica.
- APIs REST.
- Banco de dados.
- Git.
- Fundamentos de backend.
- Noções de arquitetura de software.

Profundidade esperada de um sênior

Um profissional sênior precisa saber quando usar IA, quando não usar e qual abordagem aplicar. Ele deve decidir entre regra, busca tradicional, ML clássico, RAG, fine-tuning, agente, modelo local ou API externa. Também precisa justificar tecnicamente impacto em custo, privacidade, latência, manutenção e risco.

2.2 Machine Learning clássico

O que é

Machine Learning clássico envolve algoritmos que aprendem padrões a partir de dados estruturados ou semi-estruturados. Inclui regressão linear, regressão logística, árvores de decisão, Random Forest, Gradient Boosting, XGBoost, LightGBM, SVM, KNN, K-Means e PCA.

Para que serve

Serve para problemas como:

- previsão de valores;
- classificação de categorias;
- segmentação de clientes;
- detecção de fraude;
- previsão de churn;
- score de risco;
- redução de dimensionalidade;
- análise exploratória de padrões.

Por que é importante

Mesmo com LLMs, ML clássico continua essencial porque muitos problemas empresariais são tabulares, previsíveis e mais baratos de resolver com modelos tradicionais. Um modelo XGBoost bem treinado pode ser mais barato, mais rápido e mais explicável do que usar LLM para tudo.

Principais algoritmos

Técnica	Uso principal	Visão sênior
Regressão Linear	Prever valor contínuo.	Avaliar linearidade, resíduos, multicolinearidade e interpretabilidade.
Regressão Logística	Classificação binária.	Boa baseline para problemas explicáveis e regulados.
Árvores	Decisões interpretáveis.	Úteis, mas propensas a overfitting.
Random Forest	Ensemble robusto.	Boa baseline para dados tabulares com baixo tuning.
Gradient Boosting/XGBoost/LightGBM	Alta performance tabular.	Forte em produção, mas exige controle de overfitting e tuning.
SVM	Classificação com margens.	Pode ser bom em datasets menores, mas escala pior.
KNN	Similaridade por distância.	Simples, mas caro em inferência e sensível à escala.
K-Means	Clusterização.	Exige cuidado com número de clusters e interpretação.
PCA	Redução de dimensionalidade.	Útil para compressão, visualização e ruído.

Métricas

Métrica	Quando usar
Accuracy	Classes balanceadas.
Precision	Quando falso positivo é caro.
Recall	Quando falso negativo é caro.
F1	Equilíbrio entre precision e recall.
ROC-AUC	Avaliar ranking/probabilidade em classificação binária.
MAE	Erro médio absoluto em regressão.
RMSE	Penaliza erros grandes em regressão.

Projeto prático

Modelo de classificação com pipeline completo e MLflow

Construir um projeto com:

- ingestão de dados;
- limpeza;
- feature engineering;
- split treino/validação/teste;
- pipeline com Scikit-learn;
- comparação de modelos;
- registro de métricas;
- versionamento do experimento;
- registro do modelo;
- API de inferência.

MLflow é adequado aqui porque fornece tracking de parâmetros, versões de código, métricas e artefatos, além de registry para gerenciar versões, aliases, metadata e ciclo de vida de modelos.

Critério sênior

O sênior não apenas treina o modelo. Ele define:

- métrica alinhada ao negócio;
- baseline;
- custo de falso positivo e falso negativo;
- estratégia de validação;
- risco de data leakage;
- estratégia de deploy;
- monitoramento de drift;
- plano de rollback;
- documentação técnica;
- explicabilidade mínima.

2.3 Deep Learning

O que é

Deep Learning é o uso de redes neurais profundas para aprender representações complexas. A base envolve tensors, autograd, backpropagation, loss functions, optimizers, regularization, CNNs, RNNs, LSTMs, GRUs, Transformers, embeddings e attention.

Para que serve

Serve para problemas em que os dados são complexos ou não estruturados:

- imagem;
- áudio;
- texto;

- séries temporais;
- linguagem natural;
- visão computacional;
- embeddings;
- modelos generativos;
- classificação complexa.

Conceitos obrigatórios

Conceito	Explicação técnica
Tensors	Estruturas multidimensionais usadas para representar dados e parâmetros.
Autograd	Cálculo automático de gradientes.
Backpropagation	Algoritmo que propaga erro para ajustar pesos.
Loss function	Mede o erro do modelo.
Optimizer	Atualiza pesos com base nos gradientes.
SGD/Adam	Estratégias de otimização. Adam costuma convergir mais rápido, mas nem sempre generaliza melhor.
Dropout	Regularização que reduz dependência excessiva entre neurônios.
Batch Normalization	Estabiliza treinamento normalizando ativações.
CNN	Rede forte para imagens e padrões locais.
RNN/LSTM/GRU	Redes sequenciais, importantes historicamente para texto e séries temporais.
Transformers	Arquitetura baseada em atenção, base dos LLMs modernos.
Mixed precision	Treinamento/inferência com menor precisão para performance e economia de memória.

Erros comuns

- Treinar sem dataset limpo.
- Não normalizar dados.
- Usar modelo grande demais.
- Ignorar overfitting.
- Não salvar checkpoints.
- Não separar treino e validação.
- Não medir latência de inferência.
- Confundir boa loss com bom comportamento em produção.

Projeto prático

Construir uma rede neural simples com PyTorch:

- classificação de imagens ou texto;
- loop de treino manual;
- validação;
- checkpoint;
- métricas;

- exportação do modelo;
- API de serving.

Critério sênior

Saber treinar, avaliar, salvar e servir um modelo. Além disso, saber explicar:

- por que escolheu determinada arquitetura;
- custo de GPU;
- gargalos de memória;
- batch size;
- precisão numérica;
- estratégia de inferência;
- quando usar modelo pronto;
- quando treinar do zero;
- quando usar fine-tuning;
- quando não usar Deep Learning.

2.4 LLMs

O que é

LLMs são modelos de linguagem de grande escala treinados para prever, gerar e transformar texto. Eles se baseiam principalmente na arquitetura Transformer, usando tokenização, embeddings, positional encoding, self-attention, multi-head attention, feed-forward layers, LayerNorm, pre-training, fine-tuning, instruction tuning e RLHF.

Conceitos centrais

Conceito	Explicação
Tokenização	Conversão de texto em unidades processáveis pelo modelo.
BPE	Técnica comum de subword tokenization.
Embeddings	Representações vetoriais de tokens ou textos.
Positional encoding	Informação de posição dos tokens na sequência.
Self-attention	Mecanismo que permite que tokens considerem outros tokens do contexto.
Multi-head attention	Múltiplas atenções paralelas para capturar relações diferentes.
Transformer decoder	Estrutura usada em modelos autoregressivos de geração.
Pre-training	Treinamento inicial em grandes corpora.
Fine-tuning	Ajuste do modelo para tarefa ou domínio específico.
Instruction tuning	Ajuste para seguir instruções.
RLHF	Ajuste com feedback humano para melhorar alinhamento.
Context window	Limite de tokens que o modelo consegue considerar.
Temperature	Controla aleatoriedade da geração.
Top-k/Top-p	Controlam espaço de amostragem dos próximos tokens.
Frequency/Presence penalty	Reduzem repetição e incentivam diversidade.

Conceito	Explicação
Hallucination	Geração de informação incorreta ou não fundamentada.
Grounding	Conexão da resposta com fontes, dados ou contexto verificável.

Onde é usado

- Chatbots.
- Copilotos internos.
- Análise de documentos.
- Geração de relatórios.
- Extração estruturada.
- Busca semântica.
- Atendimento automatizado.
- Revisão de código.
- Classificação textual.
- Interfaces em linguagem natural para sistemas.

Erros comuns

- Confiar cegamente na resposta do modelo.
- Não limitar escopo.
- Não usar grounding.
- Não validar saída.
- Ignorar custo por token.
- Não registrar prompts.
- Não tratar dados sensíveis.
- Não controlar alucinação.
- Não ter fallback.

Projetos práticos

1. Mini-transformer educacional

Objetivo: implementar uma versão simplificada para entender tokenização, embeddings, attention, forward pass e geração.

2. Aplicação com LLM para análise de documentos técnicos

Funcionalidades:

- upload de documentos;
- extração de texto;
- sumarização;
- perguntas e respostas;
- resposta com citação;
- validação de saída estruturada;
- logs de custo e latência.

Critério sênior

O sênior precisa saber diferenciar:

- geração livre versus resposta fundamentada;
- prompt engineering versus RAG;
- RAG versus fine-tuning;
- modelo grande versus modelo pequeno;
- API externa versus modelo local;
- qualidade percebida versus qualidade mensurada;
- protótipo versus produto observável.

2.5 Prompt Engineering avançado

O que é

Prompt Engineering é a disciplina de projetar instruções, contexto, exemplos, restrições, ferramentas e formatos de saída para obter comportamento confiável de um LLM.

Técnicas principais

Técnica	Uso
Zero-shot	Instrução sem exemplos.
Few-shot	Instrução com exemplos.
Chain-of-thought conceitual	Decomposição do problema em etapas verificáveis, sem depender de expor raciocínio interno do modelo.
ReAct	Combina raciocínio orientado a ações com uso de ferramentas.
Self-consistency	Compara múltiplas respostas para reduzir variância.
Tree of Thought	Explora alternativas de solução de forma estruturada.
Prompt templates	Padroniza prompts reutilizáveis.
Prompt versioning	Versiona mudanças e resultados.
Guardrails	Define limites, validações e bloqueios.
Structured outputs	Força saída em JSON, schema ou formato válido.
Function calling	Permite que o modelo chame funções controladas.
Tool use	Integra o modelo a APIs, busca, banco ou serviços.
Constrained decoding	Restringe o formato de geração.
Pydantic + LLM	Valida tipos, campos obrigatórios e contrato de saída.

Técnica	Uso
Prompt injection	Ataque que tenta alterar instruções ou extrair dados.
Prompt evaluation	Testa qualidade, segurança e regressão de prompts.

Por que é importante

Prompt em produção é código. Ele precisa ter:

- versão;
- testes;
- métricas;
- dono;
- changelog;
- rollback;
- avaliação;
- segurança;
- rastreabilidade.

Erros comuns

- Alterar prompt sem teste.
- Não medir regressão.
- Usar prompt gigante sem controle.
- Misturar regra de negócio com instrução textual sem versionamento.
- Não validar JSON.
- Não separar system prompt, developer prompt, contexto e entrada do usuário.
- Não proteger contra prompt injection.

Critério sênior

Saber transformar prompt em componente versionado, testável e observável. Em uma aplicação profissional, o prompt deve entrar no ciclo de engenharia como qualquer artefato crítico: passa por revisão, teste, deploy gradual e monitoramento.

2.6 RAG avançado

O que é

RAG, ou Retrieval-Augmented Generation, combina recuperação de informação com geração de texto. Em vez de depender apenas do conhecimento interno do LLM, o sistema busca documentos relevantes e fornece esse contexto para a resposta.

Pipeline técnico

- Ingestion.
- Parsing.
- Limpeza.
- Chunking.

- Chunk overlap.
- Enriquecimento com metadata.
- Geração de embeddings.
- Armazenamento em vector store.
- Busca semântica.
- Busca híbrida com BM25 + dense retrieval.
- Reranking.
- Query rewriting.
- Multi-query retrieval.
- Contextual compression.
- Controle de ACL/permissão.
- Geração com citações.
- Avaliação.
- Observabilidade.

RAGAS é um framework usado para sair de validações subjetivas e criar ciclos sistemáticos de avaliação de aplicações com LLM, incluindo avaliação de pipelines RAG.

Por que é importante

RAG resolve um problema crítico: LLMs podem não saber dados privados, internos ou atualizados. Com RAG, o sistema responde baseado em fontes controladas.

Onde é usado

- Base de conhecimento corporativa.
- Busca em artigos técnicos.
- Consulta a ADRs.
- Assistente jurídico.
- Suporte interno.
- Copiloto de engenharia.
- FAQ inteligente.
- Análise de contratos.
- Atendimento com documentos regulatórios.

Erros comuns

- Chunk muito grande ou muito pequeno.
- Embedding ruim para o domínio.
- Não usar metadata.
- Não filtrar por permissão.
- Não citar fontes.
- Recuperar contexto irrelevante.
- Não medir faithfulness.

- Não separar documentos públicos e privados.
- Não tratar documentos duplicados ou obsoletos.

Projeto prático: RAG da DevCore

Construir um RAG com:

- busca em artigos;
- busca em ADRs;
- filtro por permissão;
- citações;
- avaliação automática;
- dashboard de qualidade;
- logs de pergunta, chunks recuperados, resposta, custo e latência;
- métricas de precisão de contexto, relevância da resposta e groundedness.

Critério sênior

Um sênior deve saber decidir:

- vector search ou hybrid search;
- chunk fixo ou semântico;
- reranker local ou API;
- embedding aberto ou proprietário;
- índice por domínio ou índice unificado;
- atualização batch ou incremental;
- RAG simples ou multi-stage retrieval;
- resposta bloqueada quando não há fonte suficiente;
- estratégia de autorização por documento, chunk ou tenant.

2.7 Fine-tuning

O que é

Fine-tuning é o ajuste de um modelo pré-treinado para um domínio, formato ou comportamento específico. Pode envolver instruction dataset, limpeza de dados, split treino/validação, LoRA, QLoRA, PEFT, Hugging Face Transformers, tokenizers, quantization, avaliação, model registry e serving.

Para que serve

Serve para:

- melhorar aderência a um formato;
- adaptar vocabulário de domínio;
- reduzir dependência de prompts longos;
- especializar comportamento;
- melhorar consistência;
- reduzir custo de inferência em alguns cenários;

- criar modelo menor para tarefa específica.

Quando usar

Use fine-tuning quando:

- há dataset suficiente e de qualidade;
- o problema é repetitivo;
- o comportamento desejado é estável;
- o RAG não resolve sozinho;
- o custo de prompt/contexto está alto;
- o modelo precisa seguir estilo ou formato específico.

Quando não usar

Não use fine-tuning para:

- ensinar fatos que mudam com frequência;
- substituir base de conhecimento atualizada;
- corrigir falta de contexto;
- resolver problema de prompt mal definido;
- mascarar dataset ruim.

Projeto prático

Fine-tuning pequeno com dataset próprio de perguntas/respostas técnicas

Entregáveis:

- dataset limpo;
- split treino/validação;
- baseline com modelo base;
- fine-tuning com LoRA/QLoRA;
- avaliação comparativa;
- relatório de custo;
- registro do modelo;
- API de serving;
- comparação: base model vs fine-tuned model.

Critério sênior

O sênior precisa justificar:

- por que fine-tuning é melhor que RAG;
- custo de treinamento;
- custo de inferência;
- risco de overfitting;
- qualidade do dataset;
- risco de vazamento de dados;

- governança do modelo;
- rollback;
- estratégia de atualização.

2.8 Agentes

O que é

Agentes são sistemas em que um LLM pode planejar, escolher ferramentas, executar ações, usar memória e interagir com ambientes externos. Podem envolver tool calling, planning, memory, reflection, multi-agent systems, LangGraph, CrewAI conceitual, human-in-the-loop, sandboxing, rate limits, retry, timeout e observabilidade.

Para que serve

Serve para fluxos que exigem múltiplos passos:

- pesquisar;
- consultar APIs;
- ler documentos;
- gerar análise;
- chamar ferramenta;
- validar resultado;
- pedir aprovação humana;
- executar ação controlada.

Onde é usado

- Agente de suporte técnico.
- Agente de pesquisa documental.
- Agente de triagem de chamados.
- Agente para gerar checklist.
- Agente que consulta base interna.
- Agente que interage com ferramentas de DevOps.
- Agente de análise de logs.
- Agente de revisão de documentos.

Erros comuns

- Dar autonomia excessiva.
- Permitir acesso amplo a ferramentas.
- Não impor timeout.
- Não limitar custo.
- Não registrar ações.
- Não usar aprovação humana em ações sensíveis.
- Não isolar ambiente.
- Não validar saída da ferramenta.

- Não controlar permissões.

Projeto prático

Agente técnico controlado

O agente deve:

- pesquisar na base do blog;
- ler ADRs;
- sugerir conteúdo;
- gerar checklist;
- respeitar permissões;
- não acessar dados sem autorização;
- registrar traces;
- limitar número de chamadas;
- ter fallback;
- pedir aprovação humana em ações críticas.

Critério sênior

O sênior precisa projetar o agente como sistema distribuído e não como “chat inteligente”. Ele deve definir:

- fronteiras de autonomia;
- política de ferramentas;
- modelo de permissão;
- logs de auditoria;
- custo máximo;
- estratégia de retry;
- isolamento;
- avaliação;
- rollback;
- human-in-the-loop.

2.9 LLMOps

O que é

LLMOps é a disciplina de operar aplicações com LLMs em produção. Inclui prompt registry, model registry, experiment tracking, eval datasets, online eval, offline eval, A/B testing, canary de prompts, custo por requisição, orçamento de tokens, latência p95/p99, fallback, drift, monitoring, Langfuse, Helicone, LangSmith, OpenTelemetry, segurança e auditoria.

Por que é importante

Aplicações com LLMs são não determinísticas. Isso significa que testes tradicionais não bastam. É necessário monitorar qualidade, custo, latência, segurança, groundedness, taxa de erro, regressão de prompts e comportamento em produção.

Ferramentas como Langfuse se posicionam como plataformas de AI Engineering com tracing, prompt management, evaluation, métricas de custo/latência e integração com OpenTelemetry.

Métricas críticas

Métrica	Por que importa
Cost per request	Controla viabilidade econômica.
Token budget	Evita explosão de custo.
Latência p95/p99	Mede experiência real em produção.
Taxa de fallback	Mostra instabilidade ou baixa confiança.
Faithfulness	Mede aderência ao contexto.
Relevância	Mede utilidade da resposta.
Taxa de erro JSON	Mede confiabilidade de saída estruturada.
Taxa de bloqueio por segurança	Mede tentativas maliciosas ou falso positivo.
Drift	Detecta mudança de padrão nas entradas/saídas.

Erros comuns

- Não versionar prompt.
- Não ter dataset de avaliação.
- Não medir custo.
- Não ter rollback.
- Não comparar modelos.
- Não registrar traces.
- Não usar canary.
- Não separar avaliação offline e online.
- Não monitorar p95/p99.
- Não auditar respostas sensíveis.

Critério sênior

Saber operar aplicação de IA como produto em produção. Isso significa ter:

- rastreabilidade;
- controle de custo;
- avaliação contínua;
- segurança;
- governança;
- fallback;
- SLA/SLO;
- auditoria;
- documentação;
- processo de mudança.

2.10 Segurança e governança de IA

O que é

Segurança em IA envolve proteger a aplicação contra abuso, vazamento, manipulação e comportamento indevido. Governança envolve políticas, auditoria, documentação, avaliação de risco, revisão humana, conformidade legal e uso responsável.

Riscos principais

A OWASP mantém uma lista específica para aplicações com LLMs, incluindo riscos como prompt injection, insecure output handling, training data poisoning, model denial of service, supply chain vulnerabilities, sensitive information disclosure, insecure plugin design, excessive agency, overreliance e model theft.

Frameworks e regulações relevantes

O NIST AI RMF é um framework voluntário para melhorar a capacidade de incorporar confiança, risco e avaliação em produtos, serviços e sistemas de IA; o NIST também publicou um perfil específico para IA generativa em 2024.

O EU AI Act estabelece uma abordagem baseada em risco para desenvolvedores e deployers de IA, com foco em IA confiável, segurança, direitos fundamentais e regras específicas para usos e modelos de IA.

Segurança aplicada

Risco	Mitigação
Prompt injection	Separar instruções de dados, validar entradas, usar allowlist de ferramentas, detectar instruções maliciosas.
Data exfiltration	Filtrar contexto por permissão, mascarar dados sensíveis, bloquear saída indevida.
Jailbreak	Guardrails, políticas de recusa, testes adversariais.
Insecure tool use	Ferramentas com escopo mínimo, autorização, sandbox e human approval.
Sensitive data leakage	DLP, mascaramento, logs seguros, retenção controlada.
Output sanitization	Validar HTML, SQL, JSON, comandos e payloads antes de usar.
Model bias	Avaliação por grupos, documentação de dataset e revisão humana.
Overreliance	Exibir incerteza, citação, revisão humana e limites de uso.

Critério sênior

O sênior deve ser capaz de:

- classificar risco da aplicação;
- definir política de dados;
- decidir se pode enviar dados a APIs externas;
- desenhar modelo de permissão;
- criar trilha de auditoria;
- documentar dataset e modelo;
- aplicar LGPD;
- definir revisão humana;
- criar threat model;
- revisar arquitetura de agentes;

- impedir autonomia excessiva;
- medir risco residual.

3. Plano de estudos em 6 módulos

Módulo 1 — Fundamentos

Objetivo

Construir base conceitual, matemática e prática para entender IA, ML e avaliação.

Conteúdos

- O que é IA.
- Machine Learning.
- Deep Learning.
- NLP.
- Computer Vision.
- Generative AI.
- LLMs.
- Agentes.
- Sistemas especialistas.
- Aprendizado supervisionado.
- Não supervisionado.
- Reforço.
- Métricas.
- Overfitting.
- Generalização.
- Bias.
- Variance.

Ordem recomendada

- IA e tipos de problema.
- Estatística e métricas.
- Supervisionado/não supervisionado.
- Overfitting, bias e variance.
- NLP, visão computacional e generative AI.
- Visão geral de LLMs e agentes.

Práticas recomendadas

- Criar mapas mentais.
- Implementar exemplos simples em Python.
- Comparar problemas de classificação, regressão e clusterização.
- Fazer análise exploratória de datasets pequenos.

Exercícios técnicos

- Classificar se um problema é regressão, classificação, clusterização ou geração.
- Calcular accuracy, precision, recall e F1 manualmente.
- Explicar overfitting com curva de treino/validação.
- Criar um notebook com análise de dataset simples.

Mini projeto

Classificador simples de tickets

Entrada: título e descrição de chamados.

Saída: categoria do ticket.

Critérios de conclusão

- Você conclui o módulo quando consegue:
- explicar diferença entre IA, ML, DL e LLM;
- escolher métrica adequada para um problema;
- identificar overfitting;
- explicar bias/variance;
- montar uma baseline simples.

Nível esperado

Básico sólido.

Módulo 2 — Aplicação prática com Machine Learning clássico

Objetivo

Aprender a construir pipelines reais de ML, com treinamento, avaliação, comparação e versionamento.

Conteúdos

- Regressão linear.
- Regressão logística.
- Árvores.
- Random Forest.
- Gradient Boosting.

- XGBoost.
- LightGBM.
- SVM.
- KNN.
- K-Means.
- PCA.
- Cross-validation.
- Feature engineering.
- Feature selection.
- Pipelines.
- MLflow.
- Métricas de classificação e regressão.

Ordem recomendada

- Regressão e classificação.
- Árvores e ensembles.
- SVM/KNN/K-Means/PCA.
- Feature engineering.
- Cross-validation.
- Pipelines.
- MLflow tracking e registry.
- API de inferência.

Práticas recomendadas

- Sempre criar baseline.
- Usar pipeline para evitar data leakage.
- Versionar experimento.
- Comparar modelos por métrica de negócio.
- Separar notebook exploratório de código produtivo.

Exercícios técnicos

- Treinar 5 modelos no mesmo dataset.
- Comparar métricas.
- Criar pipeline com preprocessing.
- Registrar experimentos no MLflow.
- Expôr melhor modelo via FastAPI.

Mini projeto

API de score de risco

A API recebe dados de um cliente e retorna probabilidade de risco.

Critérios de conclusão

- Você conclui quando consegue:
- montar pipeline completo;
- explicar escolha de modelo;
- versionar experimento;
- registrar modelo;
- servir inferência;
- justificar métrica.

Nível esperado

Intermediário.

Módulo 3 — Aprofundamento em Deep Learning e LLMs

Objetivo

Entender redes neurais, Transformers e funcionamento técnico de LLMs.

Conteúdos

- Tensors.
- Autograd.
- Backpropagation.
- Loss functions.
- Optimizers.
- SGD.
- Adam.
- Regularization.
- Dropout.
- Batch normalization.
- CNN.
- RNN.
- LSTM.
- GRU.
- Transformers.
- Embeddings.
- Attention.
- Training loop.
- GPU.

- Mixed precision.
- Tokenização.
- BPE.
- Positional encoding.
- Self-attention.
- Multi-head attention.
- Pre-training.
- Fine-tuning.
- Instruction tuning.
- RLHF.
- Context window.
- Parâmetros de geração.
- Hallucination.
- Grounding.

Ordem recomendada

- PyTorch básico.
- Rede neural simples.
- CNN/RNN como base histórica.
- Attention.
- Transformer.
- Tokenização e embeddings.
- LLMs e parâmetros.
- Aplicação com LLM.

Práticas recomendadas

- Implementar treinamento manual.
- Visualizar loss.
- Salvar checkpoints.
- Testar inferência.
- Medir latência.
- Comparar temperatura e top-p.
- Testar resposta com e sem grounding.

Exercícios técnicos

- Implementar uma MLP.
- Implementar loop de treino.
- Criar embeddings simples.
- Implementar attention simplificada.
- Testar geração com diferentes temperaturas.

- Criar app de análise de documentos.

Mini projeto

Mini-transformer educacional + analisador de documentos técnicos

Critérios de conclusão

- Você conclui quando consegue:
- explicar Transformer;
- implementar partes simplificadas;
- usar LLM via API;
- controlar parâmetros de geração;
- reduzir alucinação com grounding;
- salvar e servir modelo simples.

Nível esperado

Avançado inicial.

Módulo 4 — Arquitetura, Prompt Engineering, RAG e Fine-tuning

Objetivo

Construir aplicações úteis com LLMs, combinando prompt, RAG, avaliação e fine-tuning.

Conteúdos

- Zero-shot.
- Few-shot.
- Prompt templates.
- Prompt versioning.
- Guardrails.
- Structured outputs.
- JSON mode.
- Function calling.
- Tool use.
- Pydantic + LLM.
- Prompt injection.
- Prompt evaluation.
- Ingestion.
- Parsing.
- Chunking.
- Metadata.

- Embeddings.
- Vector stores.
- Hybrid search.
- BM25.
- Dense retrieval.
- Reranking.
- Query rewriting.
- Multi-query retrieval.
- Contextual compression.
- ACL.
- Citations.
- RAG evaluation.
- Dataset design.
- LoRA/QLoRA.
- PEFT.
- Quantization.

Ordem recomendada

- Prompt engineering básico.
- Structured outputs.
- Function calling.
- RAG simples.
- RAG avançado.
- Avaliação com datasets.
- Fine-tuning básico.
- Comparação RAG vs fine-tuning.

Práticas recomendadas

- Versionar prompts.
- Validar saídas com schema.
- Exigir citações em RAG.
- Bloquear resposta sem fonte suficiente.
- Testar prompt injection.
- Separar retrieval de generation.
- Criar dataset de avaliação.
- Comparar base model, RAG e fine-tuned model.

Exercícios técnicos

- Criar prompt template versionado.
- Criar saída JSON validada com Pydantic.

- Criar RAG com vector store.
- Adicionar filtro por permissão.
- Adicionar reranking.
- Criar avaliação automática.
- Fazer fine-tuning pequeno.

Mini projeto

DevCore AI Knowledge Assistant

Funcionalidades:

- consulta artigos;
- consulta ADRs;
- responde com citação;
- respeita permissões;
- gera checklist;
- mede qualidade;
- registra custo e latência.

Critérios de conclusão

- Você conclui quando consegue:
- criar RAG confiável;
- avaliar resposta;
- controlar alucinação;
- decidir entre RAG e fine-tuning;
- validar saída estruturada;
- mitigar prompt injection básico.

Nível esperado

Avançado.

Módulo 5 — Produção, performance, segurança e escalabilidade

Objetivo

Operar aplicações de IA em produção com qualidade, custo controlado, segurança e observabilidade.

Conteúdos

- Prompt registry.
- Model registry.
- Experiment tracking.
- Eval datasets.

- Online eval.
- Offline eval.
- A/B testing.
- Canary de prompts.
- Cost per request.
- Token budget.
- Latência p95/p99.
- Fallback.
- Drift.
- Monitoring.
- Langfuse.
- Helicone.
- LangSmith.
- OpenTelemetry para LLM.
- OWASP LLM Top 10.
- Prompt injection.
- Data exfiltration.
- Sensitive data leakage.
- Output sanitization.
- Sandboxing.
- Rate limits.
- Retry.
- Timeout.
- Auditoria.

Ordem recomendada

- Observabilidade.
- Métricas de custo e latência.
- Avaliação offline.
- Avaliação online.
- Canary e A/B testing.
- Segurança.
- Auditoria.
- Escalabilidade.

Práticas recomendadas

- Tracing por requisição.
- Log de prompts e respostas com mascaramento.
- Métricas por tenant/usuário.

- Alertas de custo.
- Fallback para modelo menor.
- Circuit breaker.
- Rate limiting.
- Avaliação contínua.
- Threat modeling.

Exercícios técnicos

- Instrumentar app com tracing.
- Criar dashboard de custo.
- Medir p95/p99.
- Criar fallback.
- Testar prompt injection.
- Implementar rate limit.
- Implementar avaliação automática em batch.
- Criar canary de prompt.

Mini projeto

Plataforma LLMOps para o DevCore AI Assistant

Funcionalidades:

- prompt registry;
- eval dataset;
- tracing;
- custo por request;
- latência;
- qualidade;
- dashboard;
- alerta;
- fallback;
- auditoria.

Critérios de conclusão

- Você conclui quando consegue:
- operar app com IA em produção;
- medir custo, latência e qualidade;
- detectar regressão;
- fazer rollback;
- aplicar segurança;
- auditar decisões;
- justificar trade-offs.

Nível esperado

Avançado/sênior.

Módulo 6 — Domínio sênior e liderança técnica

Objetivo

Consolidar capacidade de arquitetura, decisão técnica, governança, liderança e evolução de produtos com IA.

Conteúdos

- Arquitetura de sistemas com IA.
- Decisão entre LLM, RAG, fine-tuning e agente.
- Design de plataforma.
- Multi-tenancy.
- Segurança por design.
- LGPD aplicada a IA.
- NIST AI RMF.
- EU AI Act conceitual.
- Model cards.
- Dataset documentation.
- Auditoria.
- Human review.
- Estratégia de custo.
- Estratégia de escalabilidade.
- Estratégia de evolução.
- Revisão técnica.
- Mentoria.
- Documentação arquitetural.

Práticas recomendadas

- Criar ADRs para decisões de IA.
- Definir padrões internos de prompt.
- Criar matriz de risco.
- Criar política de dados.
- Criar guia de uso de LLM.
- Definir critérios de aceite para IA.
- Liderar revisão de arquitetura.
- Criar roadmap técnico.

- Treinar time em segurança e avaliação.

Exercícios técnicos

- Escrever ADR: RAG vs Fine-tuning.
- Escrever ADR: API externa vs modelo local.
- Criar threat model de agente.
- Criar matriz de custo por modelo.
- Criar política de logs.
- Criar checklist de produção.
- Revisar arquitetura de outro projeto.

Mini projeto

AI Platform Blueprint

Documento arquitetural completo contendo:

- arquitetura lógica;
- arquitetura física;
- fluxo de dados;
- segurança;
- observabilidade;
- avaliação;
- custo;
- escalabilidade;
- governança;
- riscos;
- estratégia de evolução.

Critérios de conclusão

- Você conclui quando consegue:
- liderar tecnicamente um produto com IA;
- revisar decisões de outros devs;
- justificar trade-offs;
- criar arquitetura segura;
- definir padrões;
- orientar time;
- responder por produção;
- medir valor de negócio.

Nível esperado

Sênior.

4. Critérios de domínio técnico por área

IA e fundamentos

Nível	Domínio esperado
Iniciante	Entende diferença entre IA, ML, DL e LLM.
Intermediário	Aplica métricas e identifica tipos de problema.
Avançado	Modela problemas reais e escolhe abordagem adequada.
Sênior	Decide se IA é necessária, justifica risco, custo e valor.

Machine Learning

Nível	Domínio esperado
Iniciante	Treina modelos simples.
Intermediário	Cria pipeline e avalia corretamente.
Avançado	Compara modelos, evita leakage e faz tuning.
Sênior	Define métrica de negócio, governança, deploy, monitoramento e rollback.

Deep Learning

Nível	Domínio esperado
Iniciante	Entende redes neurais e treinamento.
Intermediário	Implementa loop de treino e validação.
Avançado	Ajusta arquitetura, regularização e inferência.
Sênior	Decide arquitetura, custo de GPU, estratégia de serving e manutenção.

LLMs

Nível	Domínio esperado
Iniciante	Usa APIs de LLM.
Intermediário	Controla prompts, parâmetros e saída.
Avançado	Cria aplicações com grounding, tools e avaliação.
Sênior	Define arquitetura de produto com LLM, custo, segurança, fallback e compliance.

RAG

Nível	Domínio esperado
Iniciante	Cria busca vetorial simples.
Intermediário	Implementa chunking, embeddings e resposta com contexto.
Avançado	Usa hybrid search, reranking, ACL, citations e evals.
Sênior	Projeta arquitetura multi-tenant, segura, avaliável, escalável e auditável.

Fine-tuning

Nível	Domínio esperado
Iniciante	Entende o conceito.
Intermediário	Prepara dataset simples.
Avançado	Executa LoRA/QLoRA e compara resultados.
Sênior	Decide quando fine-tuning é justificável e governa ciclo de vida do modelo.

Agentes

Nível	Domínio esperado
Iniciante	Entende tool calling.
Intermediário	Cria agente com ferramentas simples.
Avançado	Adiciona memória, retry, timeout e observabilidade.
Sênior	Define limites de autonomia, segurança, auditoria e human-in-the-loop.

LLMOps

Nível	Domínio esperado
Iniciante	Registra logs básicos.
Intermediário	Mede custo e latência.
Avançado	Versiona prompts, cria evals e dashboards.
Sênior	Opera IA como produto, com governança, SLA, rollback, segurança e melhoria contínua.

5. Aplicação prática em sistemas reais

APIs

IA pode ser exposta como API de:

- classificação;
- extração;
- sumarização;
- geração;
- resposta com documentos;
- análise semântica;
- recomendação.

Em nível sênior, a API deve ter contrato claro, validação, rate limit, autenticação, observabilidade, fallback e idempotência quando necessário.

Banco de dados

Sistemas de IA usam bancos para:

- armazenar documentos;
- registrar embeddings;

- salvar traces;
- manter histórico;
- controlar permissões;
- armazenar datasets;
- versionar avaliações.

Em RAG, geralmente há combinação de banco relacional, object storage e vector store.

Microserviços

Uma arquitetura madura pode separar:

- serviço de ingestão;
- serviço de embedding;
- serviço de retrieval;
- serviço de geração;
- serviço de avaliação;
- serviço de auditoria;
- gateway de IA;
- serviço de billing/custo.

Mensageria

Mensageria é útil para:

- ingestão assíncrona;
- reindexação;
- avaliação batch;
- geração de embeddings;
- processamento de documentos grandes;
- auditoria;
- eventos de qualidade.

Segurança

Segurança precisa existir em:

- entrada do usuário;
- montagem do prompt;
- recuperação de contexto;
- chamada de ferramentas;
- saída do modelo;
- logs;
- armazenamento;
- permissões;
- auditoria.

Observabilidade

Um sistema real deve responder:

- qual prompt foi usado?
- qual modelo respondeu?
- quais documentos foram recuperados?
- qual foi o custo?
- qual foi a latência?
- houve fallback?
- houve bloqueio de segurança?
- a resposta foi fiel ao contexto?
- qual versão do prompt/modelo estava ativa?

Testes

Testes devem incluir:

- teste unitário de templates;
- teste de schema JSON;
- teste de retrieval;
- teste de segurança;
- teste de regressão de prompt;
- teste de avaliação RAG;
- teste de carga;
- teste de custo;
- teste de fallback.

Deploy

Deploy maduro envolve:

- CI/CD;
- versionamento;
- feature flags;
- canary;
- rollback;
- variáveis seguras;
- secrets manager;
- observabilidade;
- alertas;
- aprovação para mudanças críticas.

Performance

Gargalos comuns:

- contexto grande demais;
- busca lenta;
- reranking caro;
- modelo lento;
- muitas chamadas encadeadas;
- falta de cache;
- serialização pesada;
- baixa paralelização.

Estratégias:

- cache semântico;
- modelo menor para tarefas simples;
- fallback;
- truncamento inteligente;
- compressão contextual;
- streaming;
- batch;
- filas;
- otimização de embeddings.

Manutenção evolutiva

IA exige manutenção contínua:

- atualizar prompts;
- revisar datasets;
- medir drift;
- reavaliar custos;
- revisar segurança;
- atualizar dependências;
- recalcular embeddings;
- revalidar métricas;
- remover documentos obsoletos.

6. Cronograma sugerido

Opção recomendada: 32 semanas

Fase	Semanas	Foco	Resultado
1	1-4	Fundamentos de IA e ML	Base conceitual e estatística.
2	5-8	ML clássico	Pipeline completo com MLflow.

Fase	Semanas	Foco	Resultado
3	9–13	Deep Learning	Modelo treinado, avaliado e servido.
4	14–18	LLMs e Prompt Engineering	App com LLM e saída estruturada.
5	19–23	RAG avançado	RAG com citações, ACL e avaliação.
6	24–26	Fine-tuning	Comparação base model vs fine-tuned.
7	27–29	Agentes	Agente com ferramentas, memória e segurança.
8	30–32	LLMOps, segurança e arquitetura	Produto de IA pronto para produção.

Distribuição semanal sugerida

Para cada semana:

- 40% teoria técnica;
- 40% implementação;
- 10% revisão;
- 10% documentação/ADR/checkpoint.

Checkpoints técnicos

Semana	Checkpoint
4	Explicar fundamentos, métricas, overfitting e tipos de aprendizado.
8	Entregar modelo clássico com pipeline e MLflow.
13	Treinar e servir modelo de Deep Learning simples.
18	Criar aplicação com LLM, prompts versionados e saída validada.
23	Entregar RAG com avaliação e permissões.
26	Executar fine-tuning pequeno e comparar resultados.
29	Criar agente seguro com ferramentas e limites.
32	Entregar arquitetura completa com LLMOps, segurança, custo e governança.

7. Projetos recomendados para portfólio

Projeto 1 — ML Risk Scoring API

API que recebe dados estruturados e retorna score de risco.

Deve conter:

- Scikit-learn/XGBoost;
- pipeline;
- MLflow;
- FastAPI;
- Docker;
- testes;

- métricas;
- documentação.

Projeto 2 — Document Intelligence com LLM

Aplicação que analisa documentos técnicos.

Deve conter:

- upload;
- parsing;
- sumarização;
- extração JSON;
- validação com Pydantic;
- logs;
- controle de custo.

Projeto 3 — RAG DevCore

Assistente técnico sobre artigos e ADRs.

Deve conter:

- chunking;
- embeddings;
- vector store;
- hybrid search;
- reranking;
- ACL;
- citações;
- avaliação;
- dashboard.

Projeto 4 — Fine-tuned Technical QA

Modelo ajustado para perguntas e respostas técnicas.

Deve conter:

- dataset próprio;
- limpeza;
- LoRA/QLoRA;
- comparação com modelo base;
- avaliação;
- registro do modelo.

Projeto 5 — Agente Técnico Controlado

Agente que consulta documentos e gera checklist técnico.

Deve conter:

- tool calling;
- memória curta;
- permissões;
- rate limit;
- timeout;
- human-in-the-loop;
- observabilidade.

Projeto 6 — LLMOps Dashboard

Dashboard de operação para aplicações com IA.

Deve conter:

- traces;
- custo por request;
- token usage;
- latência p95/p99;
- qualidade;
- avaliação;
- prompt registry;
- alertas.

8. Checklist de domínio técnico

Fundamentos

- Explica IA, ML, DL, NLP, LLM e agentes.
- Entende supervisionado, não supervisionado e reforço.
- Explica overfitting, generalização, bias e variance.
- Escolhe métricas corretamente.

ML clássico

- Cria pipeline de ML.
- Faz feature engineering.
- Usa cross-validation.
- Compara modelos.
- Usa MLflow.

- Serve modelo via API.

Deep Learning

- Entende tensors e autograd.
- Implementa training loop.
- Usa loss e optimizer corretamente.
- Aplica regularização.
- Salva e serve modelo.

LLMs

- Entende tokenização e attention.
- Controla temperature, top-k e top-p.
- Reduz alucinação com grounding.
- Cria aplicação com LLM.
- Valida saída estruturada.

Prompt Engineering

- Usa zero-shot e few-shot.
- Cria prompt templates.
- Versiona prompts.
- Usa function calling.
- Implementa guardrails.
- Testa prompt injection.

RAG

- Implementa ingestão.
- Define chunking.
- Usa embeddings.
- Implementa vector search.
- Implementa hybrid search.
- Usa reranking.
- Aplica ACL.
- Gera citações.
- Avalia qualidade.

Fine-tuning

- Prepara dataset.
- Faz split correto.
- Usa LoRA/QLoRA.
- Avalia modelo ajustado.
- Compara com modelo base.

- Controla custo.

Agentes

- Usa tool calling.
- Define limites de autonomia.
- Implementa retry e timeout.
- Usa sandbox.
- Aplica human-in-the-loop.
- Registra ações.

LLMOps

- Versiona prompts.
- Cria eval datasets.
- Mede custo.
- Mede p95/p99.
- Implementa fallback.
- Monitora drift.
- Faz canary.
- Audita respostas.

Segurança e governança

- Conhece OWASP LLM Top 10.
- Mitiga prompt injection.
- Protege dados sensíveis.
- Implementa output sanitization.
- Aplica LGPD.
- Documenta datasets.
- Cria model cards.
- Define revisão humana.
- Usa matriz de risco.

9. Indicadores de prontidão para nível avançado/sênior

Você está pronto para atuar em nível avançado/sênior quando conseguir:

- Projetar uma arquitetura de IA completa, não apenas consumir API de LLM.
- Justificar tecnicamente quando usar prompt, RAG, fine-tuning, agente ou ML clássico.
- Medir qualidade, não depender de opinião subjetiva.
- Controlar custo por requisição e orçamento de tokens.

- Reduzir alucinação com grounding, RAG, citações e avaliação.
- Garantir segurança contra prompt injection, vazamento de dados e uso inseguro de ferramentas.
- Operar em produção com tracing, logs, métricas, fallback e rollback.
- Criar documentação arquitetural com ADRs, riscos, trade-offs e plano de evolução.
- Mentorar outros devs em boas práticas de IA.
- Conectar IA com produto, avaliando valor, risco, custo, privacidade, performance e manutenibilidade.

Resumo final da trilha

A Trilha IA Engineering forma um profissional capaz de sair do uso superficial de IA e chegar ao domínio técnico necessário para construir produtos reais com LLMs, RAG, fine-tuning, agentes e LLMOps.

A evolução ideal é:

- Fundamentos de IA.
- Machine Learning clássico.
- Deep Learning.
- LLMs.
- Prompt Engineering.
- RAG avançado.
- Fine-tuning.
- Agentes.
- LLMOps.
- Segurança e governança.
- Arquitetura e liderança técnica.

O resultado esperado é um profissional capaz de atuar como AI Engineer avançado/sênior, com visão de engenharia, arquitetura, segurança, produto, custo, operação e governança.